Linux Detailed Issue Sheets (Group 3)

Overall index – the table below provides an overview of all the audit findings documented in this issue sheet.

Member A- Naw May (Hazel) (2403220C) Lab VM

CIS Benchmark section	Title
1.1.1	Disable unused filesystems
1.1.2	Ensure /tmp is configured
1.1.3	Ensure /var is configured
1.1.4	Ensure /var/tmp is configured
1.1.5	Ensure /var/log is configured
1.1.6	Ensure /var/log/audit is configured
1.1.7	Ensure /home is configured
1.1.8	Ensure /dev/shm is configured

Member B- Abdul Kapoor Asbar Firnas (2403219E) LAB VM

CIS Benchmark section	Title
2.2.1	Ensure xorg-x11-server-common is not installed
2.2.2	Ensure Avahi Server is not installed

2.2.3	Ensure CUPS is not installed
2.2.14	Ensure dnsmasq is not installed
2.4	Ensure nonessential services listening on the system are removed or masked
4.1.1.2	Ensure auditing for processes that start prior to auditd is enabled
4.1.1.3	Ensure audit_backlog_limit is sufficient
4.1.2.1	Ensure audit log storage size is configured
4.1.2.2	Ensure audit logs are not automatically deleted
4.1.2.3	Ensure system is disabled when audit logs are full

Member D- Haja Najumudeen Mohamed Hasif (2400251A)

CIS Benchmark section	**Title**
5.1.3	Ensure permissions on /etc/cron.hourly are configured
5.1.4	Ensure permissions on /etc/cron.daily are configured
5.1.8	Ensure cron is restricted to authorized users
5.2.7	Ensure SSH root login is disabled
5.2.12	Ensure SSH X11 forwarding is disabled
5.2.16	Ensure SSH MaxAuthTries is set to 4 or less
5.2.19	Ensure SSH LoginGraceTime is set to one minute or less
5.3.4	Ensure users must provide password for escalation
5.3.7	Ensure access to the su command is restricted

5.5.2	Ensure lockout for failed password attempts is configured
5.5.3	Ensure password reuse is limited

Member E- Loke Si Xuan (2400542C), Project VM, Section 5.6 – User Accounts and Environment, Section 6 – System Maintenance

CIS Benchmark Section	Title
5.6.1.1	Ensure password expiration is 365 days or less (Automated)
5.6.1.2	Ensure minimum days between password changes is configured (Automated)
5.6.1.4	Ensure inactive password lock is 30 days or less (Automated)
5.6.2	Ensure system accounts are secured (Automated)
5.6.4	Ensure default group for the root account is GID 0 (Automated)
5.6.5	Ensure default user umask is 027 or more restrictive
6.1.7	Ensure permissions on /etc/gshadow are configured
6.1.8	Ensure permissions on /etc/gshadow- are configured (Automated)
6.1.12	Ensure sticky bit is set on all world-writable directories (Automated)
6.1.13	Audit SUID executables (Manual)
6.1.14	Audit SGID executables (Manual)
6.1.15	Audit system file permissions (Manual)
6.2.5	Ensure no duplicate GIDs exist (Automated)
6.2.8	Ensure root PATH Integrity (Automated)

6.2.11	Ensure local interactive users own their home directories (Automated)
6.2.12	Ensure local interactive user home directories are mode 750 or more restrictive (Automated)

Member A- Naw May (Hazel) (2403220C)

Ref#	Observation(s)	Risk	Recommendation
1	**Title:** 1.1.1 Disable unused filesystems **Observation:** The system does not have explicit kernel module restrictions configured to disable unused filesystems such as cramfs, squashfs, udf, hfs, and hfsplus. **Source of information:** lsmod | egrep 'cramfs|squashfs|udf|hfs' grep -R "install <filesystem>" /etc/modprobe.d/ **Output:** No output returned. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.1	If unused filesystem kernel modules are not disabled, an attacker with sufficient privileges could load these modules to exploit vulnerabilities in legacy filesystem drivers. This increases the system's attack surface and may result in privilege escalation or kernel-level compromise.	It is recommended to disable unused filesystem kernel modules by configuring appropriate entries in the /etc/modprobe.d/ directory to prevent them from being loaded, in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.1. Any changes should be tested prior to implementation.
2	**Title:** 1.1.2 Ensure /tmp is configured **Observation:** The /tmp directory is configured as a separate filesystem on the system. However, the required mount options nodev, nosuid, and noexec are not enforced. The filesystem is mounted with default options, which do not sufficiently restrict execution or device file creation within /tmp. **Source of information:** findmnt /tmp mount | grep "on /tmp " grep -E '\s/tmp\s' /etc/fstab	If restrictive mount options are not applied to /tmp, attackers may store and execute malicious binaries or exploit setuid programs placed in this directory. This increases the risk of local privilege escalation and unauthorized code execution on the system	It is recommended to configure /tmp with the nodev, nosuid, and noexec mount options enabled, in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.2. Any changes should be tested prior to implementation.

	Output: The /tmp filesystem is mounted using default options without the nodev, nosuid, and noexec flags. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.2		
3	**Title:** 1.1.3 Ensure /var is configured **Observation:** The /var directory is configured as a separate filesystem on the system. However, it is mounted using default options and does not enforce the nodev, nosuid, and noexec mount options required by the CIS benchmark. **Source of information:** findmnt /var grep -E '\s/var\s' /etc/fstab **Output:** The /var filesystem is mounted with default options without the required restrictive flags. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.3	Without restrictive mount options on /var, attackers may exploit writable directories to store malicious files or abuse special device files. Excessive disk usage may also impact system stability and availability.	It is recommended to configure the /var filesystem with the appropriate restrictive mount options in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.3. Any changes should be tested prior to implementation.
4	**Title:** 1.1.4 Ensure /var/tmp is configured **Observation:** The `/var/tmp` directory is not configured as a separate filesystem and resides on the `/var` filesystem.	Without isolating `/var/tmp` on a separate filesystem, attackers may exploit writable temporary directories to store malicious files. Excessive usage of `/var/tmp` may also consume space	It is recommended to configure `/var/tmp` as a separate filesystem in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.4. Any changes should be tested prior to implementation.

	Source of information: `findmnt /var/tmp` `grep -E '\s/var/tmp\s' /etc/fstab` **Output:** No output returned, indicating that `/var/tmp` is not configured as a separate filesystem. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.4	required by `/var`, potentially impacting system stability and availability.	
5	**Title:** 1.1.5 Ensure /var/log is configured **Observation:** The /var/log directory is not configured as a separate filesystem and resides on the /var filesystem. **Source of information:** findmnt /var/log grep -E '\s/var/log\s' /etc/fstab **Output:** No output returned, indicating that /var/log is not configured as a separate filesystem. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.5	If /var/log is not isolated on a separate filesystem, excessive log growth may consume disk space required by /var or the root filesystem. This could lead to system instability, denial of service, or the inability to record critical security logs.	It is recommended to configure /var/log as a separate filesystem in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.5. Any changes should be tested prior to implementation.
6	**Title:** 1.1.6 Ensure /var/log/audit is configured **Observation:** The /var/log/audit directory is not configured as a separate filesystem and resides on the /var/log filesystem.	If /var/log/audit is not isolated on a separate filesystem, excessive audit log generation may consume disk space	It is recommended to configure /var/log/audit as a separate filesystem in accordance with CIS CentOS Linux 9 Benchmark

	Source of information: findmnt /var/log/audit grep -E '\s/var/log/audit\s' /etc/fstab **Output:** No output returned, indicating that /var/log/audit is not configured as a separate filesystem. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.6	required by /var/log or the root filesystem. This could prevent new audit records from being written, reducing the effectiveness of security monitoring and incident investigations.	Section 1.1.6. Any changes should be tested prior to implementation.
7	**Title:** 1.1.7 Ensure /home is configured **Observation:** The /home directory is configured as a separate filesystem on the system. However, it is mounted using default options and does not enforce the nodev, nosuid, and noexec mount options required by the CIS benchmark. **Source of information:** findmnt /home grep -E '\s/home\s' /etc/fstab **Output:** The /home filesystem is mounted with default options without the required restrictive flags. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.7	Without restrictive mount options on /home, malicious files placed within user home directories may be executed or abused for privilege escalation. Compromise of a user account may therefore have a wider impact on system security.	It is recommended to configure the /home filesystem with appropriate restrictive mount options in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.7. Any changes should be tested prior to implementation.
8	**Title:** 1.1.8 Ensure /dev/shm is configured	If the noexec option is not applied to /dev/shm, attackers may execute	It is recommended to configure /dev/shm with the nodev, nosuid, and

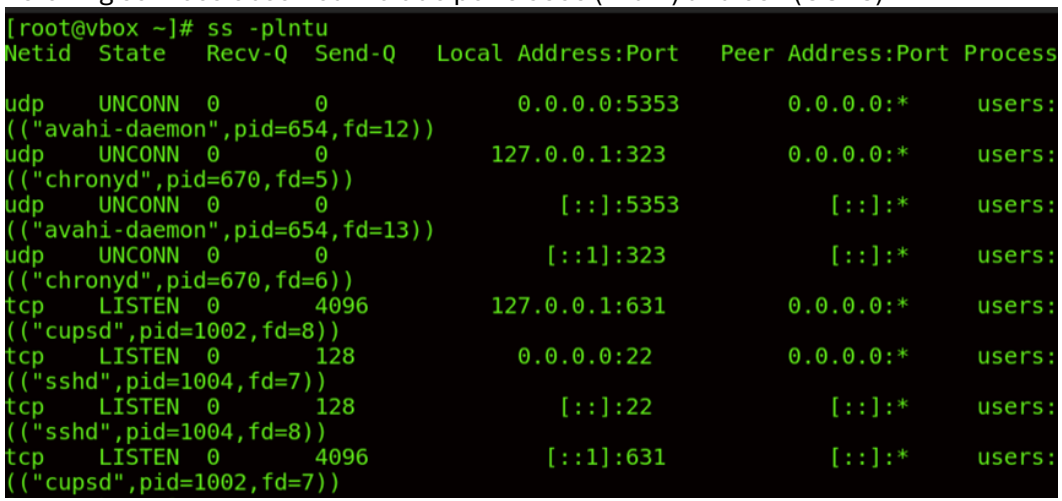
	Observation: The /dev/shm filesystem is mounted with the nodev and nosuid options. However, the noexec mount option is not enforced, and the configuration is not persistently defined in /etc/fstab. **Source of information:** findmnt /dev/shm grep -E '\s/dev/shm\s' /etc/fstab **Output:** The /dev/shm filesystem is mounted without the noexec option, and no persistent configuration was found. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 1.1.8	malicious code from shared memory locations. This increases the risk of unauthorized code execution and privilege escalation.	noexec mount options enabled in accordance with CIS CentOS Linux 9 Benchmark Section 1.1.8. Any changes should be tested prior to implementation.

Member B- Abdul Kapoor Asbar Firnas (2403219E) LAB VM

Ref#	Observation(s)	Risk	Recommendation

1	**Title: 2.2.1** Ensure xorg-x11-server-common is not installed **Observation:** The package xorg-x11-server-common is installed on the system. This was verified using the RPM package query command. **Source of information:** #rpm -q xorg-x11-server-common **Output:** xorg-x11-server-common-1.20.11-31.el9.x86_64 **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 2.2.1	The X Window System provides a graphical user interface (GUI). Servers should generally run in "headless" mode. Running a GUI increases the system's attack surface and resource usage without adding value to a dedicated server environment..	Remove the X Window System package if graphical access is not required: #dnf remove xorg-x11-server-common
2	**Title: 2.2.2** Ensure Avahi Server is not installed **Observation:** The avahi package is installed on the system, indicating that the Avahi daemon may be available. **Source of information:** #rpm -q avahi **Output:** avahi-0.8-23.el9.x86_64 **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 2.2.2	Avahi is used for service discovery (mDNS/Bonjour) on a local network. It is typically used for laptops/printers, not secure servers. It keeps network port 5353 open, allowing attackers to discover network information or spoof services.	Stop the service and remove the package if not required: #systemctl stop avahi-daemon #dnf remove avahi

3	**Title: 2.2.3** Ensure CUPS is not installed **Observation:** The Common Unix Printing System (CUPS) package is installed on the system. **Source of information:** #rpm -q cups **Output:** cups-2.3.3op2-35.el9.x86_64 **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 2.2.3	CUPS (Common Unix Printing System) is only required if the system is acting as a print server. It listens on TCP port 631. Unnecessary open ports increase the risk of remote exploitation and Denial of Service (DoS) attacks.	Remove the CUPS package if printing services are not required: #dnf remove cups
4	**Title: 2.2.14** Ensure dnsmasq is not installed **Observation:** The dnsmasq package is installed on the system. **Source of information:** #rpm -q dnsmasq **Output:** dnsmasq-2.85-17.el9.x86_64 **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 2.2.14	DNSMasq includes DHCP and DNS caching functionality. If not specifically required, running a DNS/DHCP server increases the risk of DNS cache poisoning or being used in amplification attacks.	Remove the package if DNS or DHCP services are not required: #dnf remove dnsmasq

5	**Title: 2.4** Ensure nonessential services listening on the system are removed or masked [This finding consolidates service-level controls by identifying active listening ports, which complements package-based checks under Section 2.2] **Observation(s):** Network ports associated with nonessential services are listening on the system. **Source of information**: ss -plntu **Output:** Listening services observed include ports 5353 (Avahi) and 631 (CUPS).  **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 2.4	The audit revealed ports 5353 (Avahi) and 631 (CUPS) are listening on the network interfaces. Listening services that are not required for business functions provide an entry point for attackers to exploit vulnerabilities in the service software.	Identify nonessential services and remove or mask the associated packages. Refer to removal identified in Findings 2.2.2 (Avahi) and 2.2.3 (CUPS).

6	**Title: 4.1.1.2** Ensure auditing for processes that start prior to auditd is enabled **Observation:** The kernel boot parameter audit=1 is not configured for any installed kernels. **Source of information:** #grubby --info=ALL | grep -Po '\baudit=1\b' **Output:** No output returned. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 - Section 4.1.1.2	Without the audit=1 parameter, audit events generated by processes that start before the audit daemon launches (during boot) will not be recorded. This leaves a blind spot in the logs for boot-time security events.	Enable auditing at boot for all kernels: # grubby --update-kernel ALL --args 'audit=1'
7	**Title: 4.1.1.3** Ensure audit_backlog_limit is sufficient **Observation:** The audit_backlog_limit kernel parameter is not configured. **Source of information:** # grubby --info=ALL | grep -Po "\baudit_backlog_limit=\d+\b" **Output:** No output returned. **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 4.1.1.3	If the audit buffer fills up during a surge of activity (like a boot storm or DoS attack), audit records may be dropped. This leads to a loss of accountability and forensic evidence.	Configure an appropriate backlog limit (e.g., 8192): # grubby --update-kernel ALL --args 'audit_backlog_limit=8192'

8	**Title: 4.1.2.1** Ensure audit log storage size is configured **Observation:** The audit log maximum file size is configured to 8MB. **Source of information:** # grep -w "^\s*max_log_file\s*=" /etc/audit/auditd.conf **Output**: max_log_file = 8 **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 4.1.2.1	A small audit log size may result in frequent log rotation, increasing the risk that important security events are overwritten before they can be reviewed or archived. This may negatively impact incident response and forensic analysis.	Increase the audit log size in accordance with organizational policy, for example: max_log_file = 32

9	**Title: 4.1.2.2** Ensure audit logs are not automatically deleted **Observation:** The audit system is configured to rotate (overwrite) logs when the maximum file size is reached. The parameter max_log_file_action is not set to keep_logs.Source of information: **Source of information:** # grep max_log_file_action /etc/audit/auditd.conf **Output:** max_log_file_action = ROTATE **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 4.1.2.2	If audit logs are automatically rotated or deleted when the maximum file size is reached, older security events and forensic evidence will be lost. This prevents the organization from investigating incidents that occurred in the past.	Configure the system to keep logs even when the max size is reached (administrators must manually archive them). Update /etc/audit/auditd.conf: max_log_file_action = keep_logs

10	**Title: 4.1.2.3** Ensure system is disabled when audit logs are full **Observation:** The parameter admin_space_left_action is configured as SUSPEND in /etc/audit/auditd.conf. No configuration was found where admin_space_left_action is set to halt or single, which is required by the CIS benchmark. **Source of information:** # grep space_left_action /etc/audit/auditd.conf # grep action_mail_acct /etc/audit/auditd.conf # grep -E 'admin_space_left_action\s*=\s*(halt|single)' /etc/audit/auditd.conf **Output:** space_left_action = SYSLOG action_mail_acct = root admin_space_left_action = SUSPEND **Benchmark:** CIS CentOS Linux 9 Benchmark v1.0.0 – Section 4.1.2.3	If the system continues running when audit logs can no longer be written, security-relevant events may occur without being recorded. This compromises audit integrity and reduces the ability to investigate incidents or meet compliance requirements.	Configure missing configs and the system to halt or enter single-user mode when audit logs are full by updating the following settings in /etc/audit/auditd.conf: admin_space_left_action = halt disk_full_action = halt space_left_action = email action_mail_acct = root

Member D- Haja Najumudeen Mohamed Hasif (2400251A)

Ref#	Observation(s)	Risk	Recommendation
1	**Title: 5.1.3 Ensure permissions on /etc/cron.hourly are configured** We noted that the permissions on the /etc/cron.hourly directory are set to 0755, which allows non-root users to view the contents. **Source of information:** stat /etc/cron.hourly **Output:** 0755 0 0 **Benchmark:** CIS CentOS Linux 9 Benchmark	If permissions are not restricted to root-only (0700), non-privileged users may be able to view or potentially identify vulnerabilities in scripts scheduled to run with root privileges. This provides an opportunity for local privilege escalation.	We recommend restricting directory access to the root user only. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** chown root:root /etc/cron.hourly chmod og-rwx /etc/cron.hourly

2	**Title: 5.1.4 Ensure permissions on /etc/cron.daily are configured** We noted that the permissions on the /etc/cron.daily directory are set to 0755 instead of the required 0700. **Source of information:** stat /etc/cron.daily **Output:** 0755 0 0 **Benchmark:** CIS CentOS Linux 9 Benchmark	Excessive permissions on cron directories allow unauthorized users to examine system maintenance scripts. An attacker could use this information to time attacks or find flaws in how the system handles automated tasks.	We recommend setting permissions so that only the root user can access the directory. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** chown root:root /etc/cron.daily chmod og-rwx /etc/cron.daily
3	**Title: 5.1.8 Ensure cron is restricted to authorized users** We noted that /etc/cron.deny exists while /etc/cron.allow is missing. **Source of information:** ls /etc/cron.allow ls /etc/cron.deny **Output:** /etc/cron.deny found. /etc/cron.allow is not found **Benchmark:** CIS CentOS Linux 9 Benchmark	Using a deny list instead of an allow list means any new account created on the system has cron access by default. This increases the risk of unauthorized users running scheduled background tasks or malicious scripts.	We recommend removing the deny file and creating an explicit allow list to follow a "default-deny" stance. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** rm /etc/cron.deny touch /etc/cron.allow

4	**Title: 5.2.7 Ensure SSH root login is disabled** We noted that the SSH configuration allows direct login to the "root" account. **Source of information:** sshd -T -C user=root -C host="$(hostname)" -C addr="$(grep $(hostname) /etc/hosts | awk '{print $1}')" | grep permitrootlogin **Output:** permitrootlogin yes **Benchmark:** CIS CentOS Linux 9 Benchmark	Allowing direct root login via SSH makes the system vulnerable to brute-force attacks against the most privileged account. It also hides the identity of the person logging in, removing accountability for admin actions.	We recommend disabling direct root login. Admins should login as themselves and use sudo. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Edit /etc/ssh/sshd_config: PermitRootLogin no
5	**Title: 5.2.12 Ensure SSH X11 forwarding is disabled** We noted that X11Forwarding is enabled in the SSH daemon configuration. **Source of information:** sshd -T -C user=root -C host="$(hostname)" -C addr="$(grep $(hostname) /etc/hosts | awk '{print $1}')" | grep -i x11forwarding **Output:** x11forwarding yes **Benchmark:** CIS CentOS Linux 9 Benchmark	X11 forwarding can be exploited by an attacker to "sniff" keystrokes or capture screen data from the user's local machine through the SSH tunnel, leading to the theft of sensitive information or passwords.	We recommend disabling X11 forwarding unless it is strictly required for business operations. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Edit /etc/ssh/sshd_config: X11Forwarding no

6	**Title: 5.2.16 Ensure SSH MaxAuthTries is set to 4 or less** We noted that the maximum number of authentication attempts is set to 6. **Source of information:** sshd -T -C user=root -C host="$(hostname)" -C addr="$(grep $(hostname) /etc/hosts | awk '{print $1}')" | grep maxauthtries **Output:** maxauthtries 6 **Benchmark:** CIS CentOS Linux 9 Benchmark	A higher number of attempts allows an attacker more chances to guess a password in a single connection session, increasing the probability of a successful brute-force attack against system accounts.	We recommend reducing the maximum number of authentication attempts to 4. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Edit /etc/ssh/sshd_config: MaxAuthTries 4
7	**Title: 5.2.19 Ensure SSH LoginGraceTime is set to one minute or less** We noted the login grace time is set to 120 seconds. **Source of information:** sshd -T -C user=root -C host="$(hostname)" -C addr="$(grep $(hostname) /etc/hosts | awk '{print $1}')" | grep logingracetime **Output:** logingracetime 120 **Benchmark:** CIS CentOS Linux 9 Benchmark	Long grace periods allow unauthenticated users to hold a connection open without logging in. An attacker can use this to exhaust the SSH daemon's available connections, causing a Denial of Service (DoS).	We recommend setting the login grace time to 60 seconds. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Edit /etc/ssh/sshd_config: LoginGraceTime 60

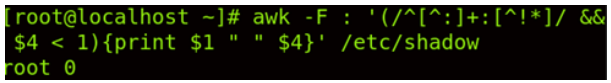
8	**Title: 5.3.4 Ensure users must provide password for escalation** We noted that sudoers entries exist allowing users to run commands as root without a password. **Source of information:** grep -r "^[^#].*NOPASSWD" /etc/sudoers* **Output:** Student ALL=(ALL) NOPASSWD: ALL **Benchmark:** CIS CentOS Linux 9 Benchmark	If a user account is compromised, the NOPASSWD setting allows the attacker to immediately execute any command as root, completely bypassing the authentication security layer.	We recommend removing the NOPASSWD flag so that users must always provide their password to escalate privileges. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Remove NOPASSWD tags from /etc/sudoers.
9	**Title: 5.3.7 Ensure access to the su command is restricted** We noted that the pam_wheel.so restriction is not active for the su command. **Source of information:** grep -Pi '^\h*auth\h+(?:required|requisite) \h+pam_wheel\.so\h+(?:[^#\n\r]+\h+)?((?!\2) (use_uid\b|group=\H+\b))\h+(?:[^#\n\r]+\h+) ?((?!\1)(use_uid\b|group=\H+\b))(\ h+.*)?$' /etc/pam.d/su **Output:** None **Benchmark:** CIS CentOS Linux 9 Benchmark	Without this restriction, any user on the system who knows the root password can use the su command to gain full privileges. This bypasses the granular access controls and superior audit logging provided by sudo, making it harder to track which individual performed specific administrative actions.	We recommend restricting su to a specific, empty group to reinforce the use of sudo. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Create an empty group according to site policy: groupadd subgroup Add to /etc/pam.d/su: auth required pam_wheel.so use_uid group=sugroup

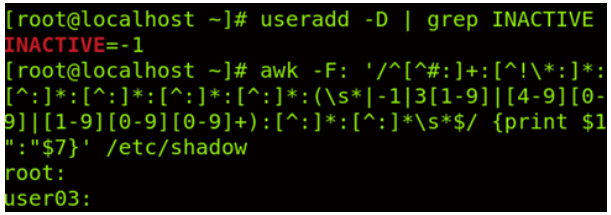
10	**Title: 5.5.2 Ensure lockout for failed password attempts is configured** We noted there is no configuration for account lockouts after failed attempts. **Source of information:** grep -E '^\s*deny\s*=\s*[1-5]\b' /etc/security/faillock.conf **Output:** None **Benchmark:** CIS CentOS Linux 9 Benchmark	Without a lockout policy, an attacker can attempt to guess passwords indefinitely using automated tools (brute-force) without the account ever being disabled.	We recommend locking accounts for 15 minutes after 5 failed attempts. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Edit /etc/security/faillock.conf: deny = 5 unlock_time = 900
11	**Title: 5.5.3 Ensure password reuse is limited** We noted that the system does not remember or restrict the reuse of previous passwords. **Source of information:** grep -P '^\h*password\h+(requisite|sufficient)\h+(pam_pwhistory\.so|pam_unix\.so)\h+([^#\n\r]+\h+)?remember=([5-9]|[1-9][0-9]+)\h*(\h+.*)?$' /etc/pam.d/systemauth **Output:** None **Benchmark:** CIS CentOS Linux 9 Benchmark	If password history is not maintained, users can immediately change their password back to an old, potentially compromised one, making password rotation requirements ineffective.	We recommend configuring the system to remember the last 5 passwords used. The following is a suggested configuration based on the benchmark. Please test it before implementing it: **Remediation:** Modify the custom system-auth file in /etc/authselect/ to include remember=5 on the pam_pwhistory.so and pam_unix.so lines. Apply changes: authselect apply-changes

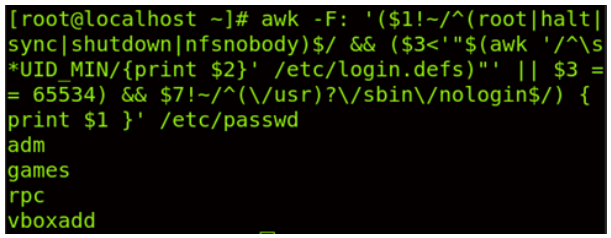
Member E- Loke Si Xuan (2400542C)

Ref#	Observation(s)	Risk	Recommendation
1	**Title:** 5.6.1.1 Ensure password expiration is 365 days or less (Automated) After reviewing the /etc/shadow file, the root user account is found to have configured the maximum password age, PASS_MAX_DAYS, to 99999 days. 99999 days means 273 years which essentially means the password will never expire. This exceeds the recommended maximum password age of 356 days or less. **Source of information:** `# grep -E '^[^:]+:[^!*]' /etc/shadow | cut -d: -f1,5` **Output:** `root:99999`	If a password is compromised through brute-force attacks, credential reuse, or malware, the attacker can continue to access the system as the system will not force out the attacker through password expiry. This increases the chances of persistent privileged access for attackers. Attackers can install backdoors or disable security controls allowing persistent access to the system. Security Risks Impacts: - Confidentiality: Attackers can read sensitive data. - Integrity: Attackers can modify or delete sensitive data. - Availability: Attackers can cause disruption to the system	To configure the password expiration to comply with the CIS recommendation, 365 days or less. Steps: 1. Edit the /etc/login.defs and set PASS_MAX_DAYS parameter to 365 days or less: `PASS_MAX_DAYS 365` 2. Apply the change to the existing non-compliant account, root: `# chage --maxdays 365 <user>`

	 **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 5.6.1.1	operations or makes the system unusable. This can lead to data breaches, system compromise, and reputation damage.	
2	**Title:** 5.6.1.2 Ensure minimum days between password changes is configured (Automated) The root user has PASS_MIN_DAYS, which is the minimum number of days have passed since the last time the user changed their password, set to 0. This is below the recommendation of 1 or more days. **Source of information:** `# awk -F : '(/^[^:]+:[^!*]/ && $4 < 1){print $1 " " $4}' /etc/shadow` **Output:** `root 0`	An attacker who gains access to the account can keep changing the password many times within a short period. This allows the attacker to reuse old or weak passwords to continue accessing the system without being detected by the system. Security Risk Impacts: - Confidentiality: Attackers can gain unauthorised access to sensitive data. - Integrity: Attackers can change configurations, weakening authentication controls.	To enforce a minimum number of days between password changes of 1 or more days. Steps: 1. Edit the /etc/login.defs and set PASS_MIN_DAYS parameter to 1 or more days: `PASS_MIN_DAYS 1` 2. Apply the change to the root account: `# chage --mindays 1 <user>`

	 **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 5.6.1.2	- Availability: Attackers can gain administrative control affecting system operations.	
3	**Title:** 5.6.1.4 Ensure inactive password lock is 30 days or less (Automated) The inactive period is set to -1 which means that accounts will never be disabled after password expiry. **Source of information:** `# useradd -D | grep INACTIVE` `# awk -F: '/^[^#:]+:[^!\*:]*:[^:]*:[^:]*:[^:]*:[^:]*:[^:]*:(\s*|-1|3[1-9]|[4-9][0-9]|[1-9][0-9][0-9]+):[^:]*:[^:]*\s*$/ {print $1":"$7}' /etc/shadow` **Output:** `INACTIVE=-1` root: user03:	Attackers may exploit inactive accounts which may go undetected as the users will not notice any anomalies or failed login attempts. Security Risk Impacts: - Confidentiality: Inactive accounts may be abused to access data. - Integrity: Unauthorised changes can be done by attackers.	Set a default inactive password lock to be 30 days or less. Steps: 1. Set the default password inactivity period to 30 days or less: `# useradd -D -f 30` 2. Modify user parameters for all users with a password set to match: `# chage --inactive 30 <user>`

	 **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 5.6.1.4		
4	**Title:** 5.6.2 Ensure system accounts are secured (Automated) Several system accounts are found to have valid interactive shells instead of nologin. The accounts are adm, games, rpc, vboxadd. **Source of information:** `# awk -F: '($1!~/^(root\|halt\|sync\|shutdown\|nfsnobody)$/ && ($3<'"$(awk '/^\s*UID_MIN/{print $2}' /etc/login.defs)"' || $3 == 65534) && $7!~/^(\/usr)?\/sbin\/nologin$/) { print $1 }' /etc/passwd`	System accounts that are not meant for regular users may still have access to an interactive shell. If these accounts are misused, an attacker can log in and execute commands on the system. Without restricting these accounts to nologin shell, they can be exploited to gain unauthorised access or perform malicious attacks. Security Risk Impacts: - Confidentiality: Unauthorised access to sensitive data. - Integrity: Unauthorised misuse of system services.	Restrict system accounts from interactive access and lock them where appropriate. Steps: **System accounts** 1. Set the shell for the system accounts found to nologin: `# usermod -s $(command -v nologin) <user>` **Disabled accounts** 2. Lock the non-root system accounts found: `# usermod -L <user>` **Large scale changes** 3. Set all system accounts to nologin: `# awk -F: '($1!~/^(root\|halt\|sync\|shutdown\|nfsnobody)$/ && ($3<'"$(awk '/^\s*UID_MIN/{print $2}' /etc/login.defs)"' || $3 == 65534)) { print $1 }' /etc/passwd | while read user; do usermod -s $(command -v nologin) $user >/dev/null; done` Lock all accounts that have their shell set to nologin:

	Output: adm games rpc vboxadd  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 5.6.2	- Availability: Abuse of system services can lead to service disruptions.	# awk -F: '/nologin/ {print $1}' /etc/passwd | while read user; do usermod -L $user; done
5	**Title:** 5.6.4 Ensure default group for the root account is GID 0 (Automated) The root account group ID, GID, is not set to 0 but 1002. **Source of information:** # grep "^root:" /etc/passwd | cut -f4 -d:	Using GID 0 helps to reduce the risk of improper file ownership and access permissions. This also helps to ensure that root-owned files will not accidentally be accessible for non-privileged users. Security Risk Impacts:	Configure root account GID to 0. Steps: 1. Set root account GID to 0 # usermod -g 0 root

	Output: 1002  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 5.6.4	- Confidentiality: Unauthorised access to root-owned files. - Privilege Boundary Weakened: Weakened privilege separation and increased likelihood of permission misconfigurations.	
6	**Title:** 5.6.5 Ensure default user umask is 027 or more restrictive (Automated) The umask is set to 022, which is less restrictive than the recommended 027. **Source of information:** # grep -RPi '(^|^[^#]*)\s*umask\s+([0-7][0-7][01][0-7]\b|[0-7][0-7][0-7][0-6]\b|[0-7][01][0-7]\b|[0-7][0-7][0-6]\b|(u=[rwx]{0,3},)?(g=[rwx]{0,3},)?o=[rwx]+\b|(u=[rwx]{1,3},)?g=[^rx]{1,3}(,o=[rwx]{0,3})?\b)' /etc/login.defs /etc/profile* /etc/bashrc* #!/bin/bash passing="" grep -Eiq '^\s*UMASK\s+(0[0-7][2-7]7|[0-7][2-7]7)\b' /etc/login.defs && grep -Eqi '^\s*USERGROUPS_ENAB\s*"?no"?\b'	A less restrictive umask allow attackers using a low privilege to read or write files. This can result in accidental data being exposed. Security Risk Impacts: - Confidentiality: Data being leaked and accessible to attackers. - Integrity: Unauthorised modification of files.	Set a more restrictive umask of 027 or better. Steps: 1. Review /etc/bashrc, /etc/profile, and all files ending in *.sh in the /etc/profile.d/ directory 2. Remove or edit the umask entries to follow local site policy, 027 or more restrictive. 3. Configure umask in one of the following files: • A file ending in .sh in the /etc/profile.d/ directory • /etc/profile • /etc/bashrc # vi /etc/profile.d/set_umask.sh umask 027 4. Run and remove or modify the umask of any returned files: # grep -RPi '(^|^[^#]*)\s*umask\s+([0-7][0-7][01][0-7]\b|[0-7][0-7][0-7][0-6]\b|[0-7][01][0-7]\b|[0-7][0-7][0-6]\b|(u=[rwx]{0,3},)?(g=[rwx]{0,3},)?o=[rwx]+\b|(u=[rwx]{1,3},)?g=[^rx]{1,3}(

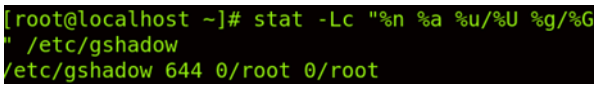
	/etc/login.defs && grep Eq'^\s*session\s+(optional|requisite|required)\s+pam_umask\.so\b'/etc/pam.d/common-session && passing=true grep -REiq '^\s*UMASK\s+\s*(0[0-7][2-7]7|[0-7][2-7]7|u=(r?|w?|x?)(r?|w?|x?)(r?|w?|x?),g=(r?x?|x?r?),o=)\b' /etc/profile*/etc/bashrc* && passing=true ["$passing" = true] && echo "Default user umask is set" **Output:** /etc/profile.d/umask.sh:umask 022 /etc/bashrc: [`umask` -eq 0] && umask 022 None		,o=[rwx]{0,3})?\b)' /etc/login.defs /etc/profile* /etc/bashrc* 5. Edit /etc/login.defs and change the UMASK and USERGROUPS_ENAB lines: UMASK 027 USERGROUPS_ENAB no 6. Edit /etc/pam.d/password-auth and /etc/pam.d/system-auth and add or edit the following: session optional pam_umask.so OR 7. Configure umask in one of the following files: • A file in the /etc/profile.d/ directory ending in .sh • /etc/profile • /etc/bashrc Example: /etc/profile.d/set_umask.sh umask 027 Note: step 7 method only applies to bash and shell. If other shells are supported on the system, it is recommended that their configuration files also are checked.

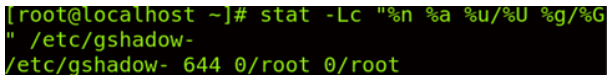
```
[root@localhost ~]# cat clause565.sh
#!/bin/bash

passing=""
grep -Eiq '^\s*UMASK\s+(0[0-7][2-7]7|[0-7][2-7]
7)\b' /etc/login.defs && grep -Eqi '^\s*USERGRO
UPS_ENAB\s*"?no"?\b' /etc/login.defs && grep -E
q '^\s*session\s+(optional|requisite|required)\
s+pam_umask\.so\b' /etc/pam.d/system-auth && pa
ssing=true
grep -REiq '^\s*UMASK\s+\s*(0[0-7][2-7]7|[0-7][
2-7]7|u=(r?|w?|x?)(r?|w?|x?)(r?|w?|x?),g=(r?x?|
x?r?),o=)\b' /etc/profile* /etc/bashrc* && pass
ing=true
[ "$passing" = true ] && echo "Default user uma
sk is set"
```

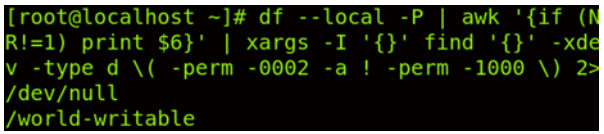
```
[root@localhost ~]# grep -RPi '(^|^[^#]*)\s*uma
sk\s+([0-7][0-7][01][0-7]\b|[0-7][0-7][0-7][0-6
]\b|[0-7][01][0-7]\b|[0-7][0-7][0-6]\b|(u=[rwx]
{0,3},)?(g=[rwx]{0,3},)?o=[rwx]+\b|(u=[rwx]{1,3
},)?g=[^rx]{1,3}(,o=[rwx]{0,3})?\b)' /etc/login
.defs /etc/profile* /etc/bashrc*
/etc/profile.d/umask.sh:umask 022
/etc/bashrc:    [ `umask` -eq 0 ] && umask 022
[root@localhost ~]# ./clause565.sh
```

Benchmark:

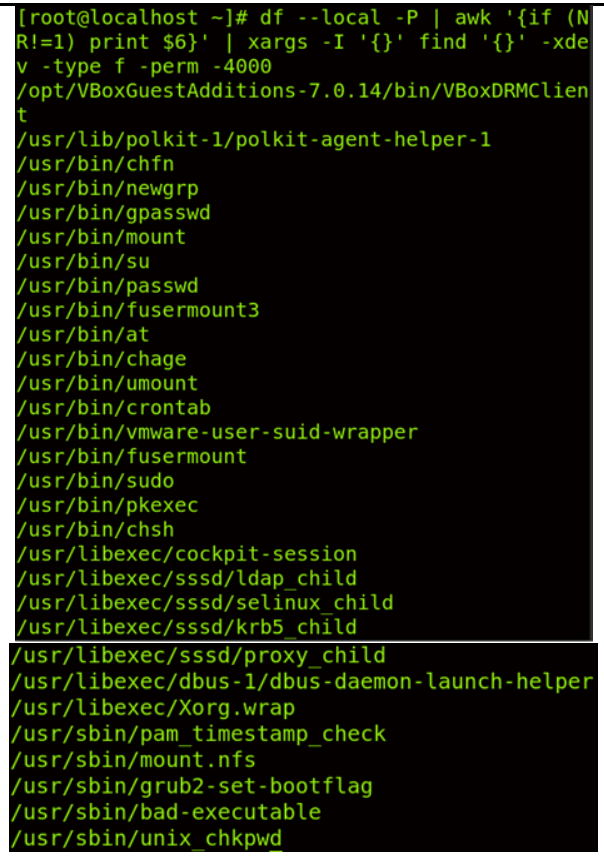
CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 5.6.5

7	**Title:** 6.1.7 Ensure permissions on /etc/gshadow are configured (Automated) The /etc/gshadow file has file permissions set to 644 which allows non-privileged users to read the file. **Source of information:** `# stat -Lc "%n %a %u/%U %g/%G" /etc/gshadow` **Output:** `/etc/gshadow 644 0/root 0/root`  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.1.7	Non-privileged users can read the /etc/shadow file if the file permissions are not set appropriately. This can lead to attackers viewing the file where hashed group passwords and administrative group information is exposed. This can be exploited for password cracking. Security Risk Impacts: - Confidentiality: Sensitive authentication and administrative group information is exposed. - Integrity: Group privilege manipulation.	Configure /etc/gshadow to be restricted. Steps: 1. Set mode, owner, and group on /etc/gshadow: `# chown root:root /etc/gshadow` `# chmod 0000 /etc/gshadow`

8	**Title:** 6.1.8 Ensure permissions on /etc/gshadow- are configured (Automated) The backup /etc/gshadow- file has file permissions set to 644 which can expose sensitive information. **Source of information:** `# stat -Lc "%n %a %u/%U %g/%G" /etc/gshadow-` **Output:** `/etc/gshadow- 644 0/root 0/root`  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.1.8	The /etc/gshadow- is the backup file of /etc/gshadow. The backup file can be viewed by non-privileged users, exposing sensitive information. Attackers can use password cracking to crack the hashed passwords, finding the credentials. Security Risk Impacts: - Confidentiality: Exposure of backup authentication data. - Privilege Escalation: The information leaked may be used to gaining elevated access.	Configure /etc/gshadow- to be restricted. Steps: 1. Set mode, owner, and group on /etc/gshadow-: `# chown root:root /etc/gshadow-` `# chmod 0000 /etc/gshadow-`

9	**Title:** 6.1.12 Ensure sticky bit is set on all world-writable directories (Automated) The directory /world-writable does not have the sticky bit set. **Source of information:** df --local -P | awk '{if (NR!=1) print $6}' | xargs -I '{}' find '{}' -xdev -type d \(-perm -0002 -a ! -perm -1000 \) 2>/dev/null **Output:** /world-writable  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.1.12	Without the sticky bit, any user can rename or delete files owned by other users in the directory. Security Risk Impacts: - Integrity: unauthorised deletion or modification of files. - Availability: Shared directories is disrupted.	Apply sticky bit on all world-writable directories. Steps: 1. Set the sticky bit on all world writable directories: # df --local -P | awk '{if (NR!=1) print $6}' | xargs -I '{}' find '{}' -xdev -type d \(-perm -0002 -a ! -perm -1000 \) 2>/dev/null | xargs -I '{}' chmod a+t '{}'
10	**Title:** 6.1.13 Audit SUID executables (Manual) An unexpected SUID binary was found, /usr/sbin/bad-executable. Despite the fact that	SUID binaries execute with elevated privileges. A malicious SUID binary can allow attackers to gain higher elevated privileges.	Ensure and check that no unexpected or rouge SUID programs have been added into the system. Review and confirm the integrity of the binaries found. Steps:

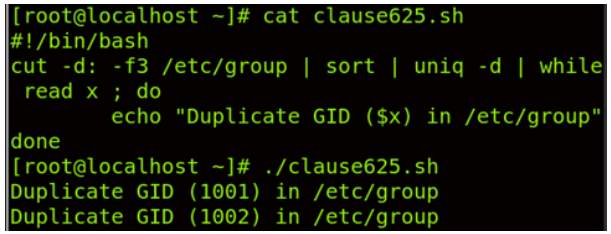
	/usr/sbin is a trusted directory, the SUID binaries should still be reviewed to ensure that they are required and authorised. **Source of information:** `# df --local -P | awk '{if (NR!=1) print $6}' | xargs -I '{}' find '{}' -xdev -type f -perm -4000` **Output:** `/usr/sbin/bad-executable`	Security Risk Impacts: - Confidentiality: System compromised. - Integrity: Unauthorised system changes. - Availability: System is misused which can cause services to be disrupted.	1. If the binaries are not legitimate or required, remove the SUID bit. `# chmod u-s <file>`

	 **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.1.13		
11	**Title:** 6.1.14 Audit SGID executables (Manual)	SGID executables can grant elevated group privileges, allowing attackers to exploit	Ensure and check that no unexpected or rouge SGID executables added into the system. Review and confirm the integrity of the binaries found.

	An unexpected SGID binary was found, /usr/sbin/bad-executable. Despite the fact that /usr/sbin is a trusted directory, the SGID binaries should still be reviewed to ensure that they are required and authorised. **Source of information:** `# df --local -P | awk '{if (NR!=1) print $6}' | xargs -I '{}' find '{}' -xdev -type f -perm -200` **Output:** `/usr/sbin/bad-executable`  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.1.14	this to access restricted group resources. Security Risk Impacts: - Confidentiality: Unauthorised data access. - Integrity: Unauthorised modification of data and abuse of group privileges.	Steps: 1. If the files are not legitimate or required, remove the SGID bit. `# chmod g-s <file>`
12	**Title:** 6.1.15 Audit system file permissions (Manual)	Packaged system files and directories should be maintained with the permissions intended by the	Correct any discrepancies found and run the audit output until clean or risk is mitigated or accepted.

	The file /etc/at.deny is missing from the system. **Source of information:** # rpm -Va --nomtime --nosize --nomd5 --nolinkto > clause6_1_15.txt # cat clause6_1_15.txt **Output:** missing c /etc/at.deny  **Benchmark:**	OS vendor. The missing system files indicates tampering or incomplete package installation, weakening access controls. Security Risk Impacts: - Integrity: Deviation from the system baseline. - Availability: Scheduled tasked misuse can affect services.	Steps: 1. Restore the file from the package. # dnf reinstall <package>

	CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.1.15		
13	**Title:** 6.2.5 Ensure no duplicate GIDs exist (Automated) Duplicated GIDs, group IDs, was found in /etc/group, 1001, 1002. **Source of information:** Create a bash file: `# nano clause625.sh` Enter following to the created bash file: `#!/bin/bash` `cut -d: -f3 /etc/group | sort | uniq -d | while read x ; do` `  echo "Duplicate GID ($x) in /etc/group"` `done` Make the bash file executable and run it: `# chmod +x clause625.sh` `# ./clause625.sh` **Output:**	GIDs should be unique to ensure accountability and appropriate access protections. This helps to ensure file ownership and access control is clear. Security Risk Impacts: - Confidentiality: Unintended access. - Integrity: Incorrect permission enforcement.	Establish unique GIDs and review the files owned by the shared GID to determine which group they belong to. Steps: 1. Change the GID of an existing group `# groupmod -g <new_gid> <group>` 2. Find all the files that belong to the old GID and change them to the correct group. `# find / -group <old_gid> -exec chgrp <new_group> {} \;`

	Duplicated GID (1001) in /etc/group Duplicated GID (1002) in /etc/group  **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.2.5		
14	**Title:** 6.2.8 Ensure root PATH Integrity (Automated) The root PATH contains the current working directory (.). **Source of information:** Create a bash file: # nano clause628.sh Enter following to the created bash file: # !/bin/bash	Attackers may place malicious executables in writable directories that are executed unintentionally by root. Security Risk Impacts: - Confidentiality: Privileged data exposed. - Integrity: Command hijacking.	Correct or justify any items discovered. Steps: 1. Remove unsafe PATH entries. # vi <PATH entry> 2. Remove (.) from PATH and reload. # source <PATH entry>

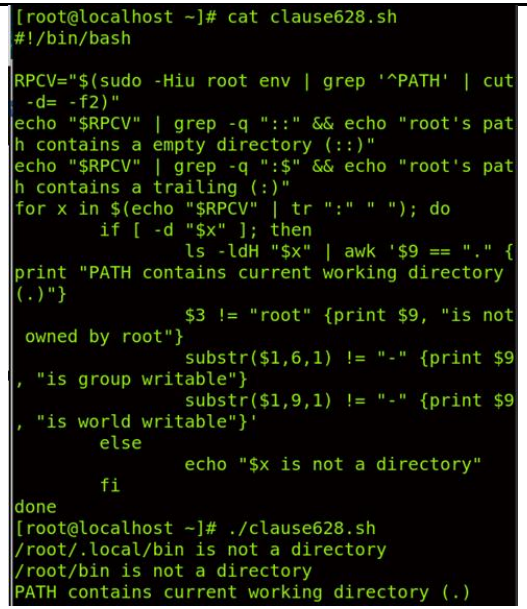
```
RPCV="$(sudo -Hiu root env | grep '^PATH' | cut -d= -
f2)"
echo "$RPCV" | grep -q "::" && echo "root's path
contains a empty directory (::)"
echo "$RPCV" | grep -q ":$" && echo "root's path
contains a trailing (:)"
for x in $(echo "$RPCV" | tr ":" " "); do
  if [ -d "$x" ]; then
    ls -ldH "$x" | awk '$9 == "." {print "PATH contains
current working directory (.)"}
    $3 != "root" {print $9, "is not owned by root"}
    substr($1,6,1) != "-" {print $9, "is group writable"}
    substr($1,9,1) != "-" {print $9, "is world writable"}'
  else
    echo "$x is not a directory"
  fi
done
```

Make the bash file executable and run it:

```
# chmod +x clause628.sh

# ./clause628.sh
```

Output:

```
/ root/.local/bin is not a directory

/root/bin is not a directory

PATH contains current working directory (.)
```

	 **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.2.8		

<table>
<tr>
<td>15</td>
<td>Title:

6.2.11 Ensure local interactive users own their home directories (Automated)

Some user home directories are owned by incorrect users. The users home directories, adm, games, user02, are owned by the wrong users.

Source of information:

Create a bash file:

nano clause6211.sh

Enter following to the created bash file:

#!/usr/bin/env bash

{
output=""
valid_shells="^($(sed -rn '/^\//{s,/,\\\\/,g;p}' /etc/shells |
paste -s -d '|' -))$"
awk -v pat="$valid_shells" -F: '$(NF) ~ pat { print $1 " " $(NF-1) }' /etc/passwd | (while read -r user home; do
owner="$(stat -L -c "%U" "$home")"
["$owner" != "$user"] && output="$output\n - User \"$user\" home
directory \"$home\" is owned by user \"$owner\""
done
if [-z "$output"]; then</td>
<td>Users need to be accountable for their home directories to ensure there is no unauthorised access.

Security Risk Impacts:

- Confidentiality: Data exposed to the wrong user.
- Integrity: Unauthorised modification.</td>
<td>Change and correct the ownership of any home directories that are not owned by the defined user to the correct user.

Steps:

1. Update user home directories to be owned by the user:
#!/usr/bin/env bash

{
output=""
valid_shells="^($(sed -rn '/^\//{s,/,\\\\/,g;p}' /etc/shells | paste -s -d '|' -))$"
awk -v pat="$valid_shells" -F: '$(NF) ~ pat { print $1 " " $(NF-1) }' /etc/passwd | while read -r user home; do
owner="$(stat -L -c "%U" "$home")"
if ["$owner" != "$user"]; then
echo -e "\n- User \"$user\" home directory \"$home\" is owned by user \"$owner\"\n - changing ownership to \"$user\"\n"
chown "$user" "$home"
fi
done
}</td>
</tr>
</table>

	echo -e "\n-PASSED: - All local interactive users have a home directory\n" else echo -e "\n- FAILED:\n$output\n" fi) } Make the bash file executable and run it: # chmod +x clause6211.sh # ./clause6211.sh **Output:** - FAILED: - User “adm” home directory “/var/adm” is owned by user “root” - User “games” home directory “/usr/games” is owned by user “root” - User “user02” home directory “/home/user02” is owned by user “user03”		

	 **Benchmark:** CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.2.11		
16	**Title:** 6.2.12 Ensure local interactive user home directories are mode 750 or more restrictive (Automated)	Malicious users, like attackers, can exploit or steal or modify other user's data or	A monitoring policy to be established. The policy will report user file permissions and determine the action to be taken. Permissions should also be restricted to 750 or more restrictive.

	Serval home directories have permissions set to 755, which allows other users to access. **Source of information:** Create a bash file: `# nano clause6212.sh` Enter following to the created bash file: `# !/usr/bin/env bash` `{` `  output=""` `  perm_mask='0027'` `  maxperm="$( printf '%o' $(( 0777 &~$perm_mask)) )"` `  valid_shells="^($( sed -rn'/^\//{s,/,\\\\/,g;p}' /etc/shells \|` `paste -s -d '\|' - ))$"` `  awk -v pat="$valid_shells" -F: '$(NF) ~ pat { print $1 " " $(NF-1) }' /etc/passwd \| (while read -r user home; do` `  mode=$( stat -L -c '%#a' "$home" )` `  [ $(( $mode & $perm_mask )) -gt 0 ] && output="$output\n- User $user home directory: \"$home\" is too permissive: \"$mode\" (should be:\"$maxperm\" or more restrictive)"` `  done` `  if [ -n "$output" ]; then` `  echo -e "\n- Failed:$output"` `  else`	gain another user's system privileges. Security Risk Impacts: - Confidentiality: Data leakage. - Integrity: Unauthorised file modification.	Steps: 1. Restrict permissions to 750 or more restrictive from user home directories: `#!/usr/bin/env bash` `{` `  perm_mask='0027'` `  maxperm="$( printf '%o' $(( 0777 & ~$perm_mask)) )"` `  valid_shells="^($( sed -rn '/^\//{s,/,\\\\/,g;p}' /etc/shells \| paste -s -d '\|' - ))$"` `  awk -v pat="$valid_shells" -F: '$(NF) ~ pat { print $1 " " $(NF-1) }' /etc/passwd \| (while read -r user home; do` `  mode=$( stat -L -c '%#a' "$home" )` `  if [ $(( $mode & $perm_mask )) -gt 0 ]; then` `  echo -e "- modifying User $user home directory: \"$home\"\n- removing excessive permissions from current mode of \"$mode\""` `  chmod g-w,o-rwx "$home"` `  fi` `  done` `  )` `}`

	echo -e "\n- Passed:\n- All user home directories are mode:\"$maxperm\" or more restrictive" fi) } Make the bash file executable and run it: # chmod +x clause6212.sh # ./clause6212.sh **Output:** - FAILED: - User adm home directory: “/var/adm” is too permissive: “0755” (should be: “750” or more restrictive) - User games home directory: “/usr/games” is too permissive: “0755” (should be: “750” or more restrictive) - User rpc home directory: “/var/lib/rpcbind” is too permissive: “0755” (should be: “750” or more restrictive)		

```
[root@localhost ~]# cat clause6212.sh
#!/usr/bin/env bash
{
        output=""
        perm_mask='0027'
        maxperm="$( printf '%o' $(( 0777 & ~$pe
rm_mask)) )"
        valid_shells="^($( sed -rn '/^\//{s,/,\
\\\/,g;p}' /etc/shells | paste -s -d '|' - ))$"
        awk -v pat="$valid_shells" -F: '$(NF) ~
 pat { print $1 " " $(NF-1) }' /etc/passwd | (w
hile read -r user home; do
                mode=$( stat -L -c '%#a' "$home
" )
                [ $(( $mode & $perm_mask )) -gt
 0 ] && output="$output\n- User $user home dire
ctory: \"$home\" is too permissive: \"$mode\" (
should be:\"$maxperm\" or more restrictive)"
        done
        if [ -n "$output" ]; then
                echo -e "\n- Failed:$output"
        else
                echo -e "\n- Passed:\n- All use
r home directories are mode:\"$maxperm\" or mor
e restrictive"
        fi
        )
}
```

```
[root@localhost ~]# ./clause6212.sh

- Failed:
- User adm home directory: "/var/adm" is too pe
rmissive: "0755" (should be:"750" or more restr
ictive)
- User games home directory: "/usr/games" is to
o permissive: "0755" (should be:"750" or more r
estrictive)
- User rpc home directory: "/var/lib/rpcbind" i
s too permissive: "0755" (should be:"750" or mo
re restrictive)
```

Benchmark:

CIS Red Hat Enterprise Linux 9 Benchmark v1.0.0 clause 6.2.12